

Instance Format Specification

1. Filename

Each paired scenario uses the same filename:

```
MPVRP_A_sB_dC_pD.dat
```

Field	Meaning
A	Instance number, from 001 to 150 for the benchmark
B	Number of service stations
C	Number of depots
D	Number of products

Example:

```
MPVRP_001_s48_d1_p1.dat
```

The repository contains two directories with a one-to-one filename mapping:

- `with_changeover_costs`: official instances used by the evaluator;
- `without_changeover_costs`: comparative twins whose transition matrices contain only zeroes.

All other data, including the UUID, fleet, coordinates, stocks and demands, is identical inside a pair.

2. Parser Rules

The LP parser tokenizes the complete file. To stay compatible:

- The first non-empty line must be the UUID comment line.
- Do not add any other comment line anywhere in the file.
- Blank lines should be avoided.
- Values may be separated by spaces or tabs.
- Entity IDs are one-based and contiguous: `1, ..., n`.
- Product IDs are one-based: `1, ..., NbProducts`.

3. File Blocks

The block order is fixed:

```
# <uuid>
NbProducts NbDepots NbGarages NbStations NbVehicles
<NbProducts rows of transition costs>
<NbVehicles rows of vehicles>
<NbDepots rows of depots>
<NbGarages rows of garages>
<NbStations rows of stations>
```

The expected number of data lines after the UUID line is:

```
1 + NbProducts + NbVehicles + NbDepots + NbGarages + NbStations
```

4. UUID

Line 1 contains a UUID comment:

```
# c01ab718-9a2c-4a7d-bb95-f37e2a389409
```

This line is mandatory for compatibility with `MPVRPInstance.read()`.

5. Global Parameters

Line 2 contains five positive integers:

```
NbProducts NbDepots NbGarages NbStations NbVehicles
```

Example:

```
3 2 1 20 5
```

6. Transition Cost Matrix

Next come `NbProducts` rows, each with `NbProducts` numeric values:

```
Cost_P1_to_P1 Cost_P1_to_P2 ...
Cost_P2_to_P1 Cost_P2_to_P2 ...
...
```

Requirements:

- Costs must be finite and non-negative.
- The diagonal must be zero.
- The matrix may be asymmetric. The solver uses

`cost[previous_product - 1][next_product - 1]`.

7. Vehicles

Next come `NbVehicles` rows:

```
ID Capacity HomeGarage InitialProduct
```

Requirements:

- `ID` must be unique and contiguous in `[1, NbVehicles]`.
- `Capacity` must be strictly positive.
- `HomeGarage` must reference an existing garage ID.
- `InitialProduct` must be in `[1, NbProducts]`.

8. Depots

Next come `NbDepots` rows:

```
ID X Y Stock_P1 Stock_P2 ... Stock_Pn
```

Requirements:

- `ID` must be unique and contiguous in `[1, NbDepots]`.
- Coordinates must be finite.
- Stocks must be finite and non-negative.
- For each product, total depot stock must be at least total station demand.

9. Garages

Next come `NbGarages` rows:

```
ID X Y
```

Requirements:

- `ID` must be unique and contiguous in `[1, NbGarages]`.
- Coordinates must be finite.

10. Service Stations

Next come `NbStations` rows:

```
ID X Y Demand_P1 Demand_P2 ... Demand_Pn
```

Requirements:

- `ID` must be unique and contiguous in `[1, NbStations]`.
- Coordinates must be finite.
- Demands must be finite and non-negative.
- Each station must have at least one positive demand.
- For LP compatibility, each station/product demand must not exceed the sum of all vehicle capacities.
The LP allows split delivery across vehicles, but it limits a vehicle to at most one visit for the same station/product pair.

11. Complete Example

```
# c01ab718-9a2c-4a7d-bb95-f37e2a389409
2 1 2 3 2
0.0 18.1
61.5 0.0
1 20000 1 1
2 20000 1 2
1 81.6 63.6 57914 82626
1 98.1 49.6
2 56.8 26.0
1 23.5 42.2 0 4278
2 3.5 38.3 1344 2350
3 56.7 31.3 0 2319
```

This instance has 2 products, 1 depot, 2 garages, 3 stations, and 2 vehicles.